LLMGraph: Autonomous Graph Learning with LLMs through a Knowledge-Driven Approach

Xiaohan Zheng[1], **Lanning Wei**[2], **Huan Zhao**[3], **Yong Li**[1], **Quanming Yao**[1]

[1]Department of Electronic Engineering, Tsinghua University [2]Ant Group [3] Noumena AI

Abstract

While graph-structured data is ubiquitous in data science, autonomous learning from graphs is a hard problem. Specifically, it requires learning methods can simultaneously understand intention of tasks and topological structure over graphs. However, none of existing approaches can address this issue. In this paper, we propose LLMGraph, which can empirically solve the issue. Through a knowledge-driven approach, LLMGraph enables its agents to interact with knowledge bases to derive deep insights into tasks and graphs, leveraging the task analysis capabilities of Large Language Models (LLMs) and the graph processing capabilities of graph learning methods to achieve autonomous graph learning.

Code and extended version —
https://github.com/lgssstsp/LLMGraph.git

Introduction and Related Work

Graph-structured data play a crucial role in various domains such as chemistry, e-commerce, and social networks (Gilmer et al. 2017; Zhang et al. 2023; Lü et al. 2012), displaying extensive variability and complexity. This characteristics necessitate autonomous graph learning (LeCun 2022) solutions capable of simultaneously understanding diverse tasks and graphs.

However, existing methods cannot fully satisfy these requirements. A variety of methods, such as Graph Neural Networks (GNNs) (Fatemi, Halcrow, and Perozzi 2023; Wang et al. 2024; Li et al. 2023), have been customized to analyze and handle graph-structured data. While these approaches are effective in processing these complex data, they often lack the ability to grasp the underlying intentions of the tasks they are applied to. On the other hand, large language models (LLMs) have shown strong ability to handle diverse tasks because of their embedded domain knowledge (Brown et al. 2020). LLMs handle graph learning tasks by flattening graphs into natural language (Fatemi, Halcrow, and Perozzi 2023; Wang et al. 2024; Li et al. 2023) or tuning models directly on graphs (Sun et al. 2022; Liu et al. 2023). However, these methods struggle with a comprehensive understanding of diverse graph structures, particularly when adapting to large-scale graphs.

Copyright © 2025, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Therefore, we introduce LLMGraph, a solution designed to achieve autonomous graph learning through a knowledge-driven approach that leverages the task analysis capabilities of LLMs and the graph processing capabilities of graph learning methods. Specifically, these agents are meticulously engineered to interact with knowledge bases to meet task-specific requirements, and handle the entire graph learning pipeline. Lastly, we showcase the practical applications of LLMGraph, demonstrating its feasibility and effectiveness in handling real-world problems.

System Architecture

As shown in Figure 1, each agent in LLMGraph interacts with knowledge bases to effectively execute the graph learning pipeline. The two types of knowledge bases, generated by the Knowledge Agent, play a pivotal role in LLMGraph. They bridge the gap between theoretical research and practical application, enabling agents to gain a deeper understanding of graphs and tasks and make better decisions in a knowledge-driven manner. We now briefly describe each agent and how they collaborate within LLMGraph. For more details, please check the appendix of extended version.

Planning Agent The Planning Agent generate a task plan based on user instructions, to direct subsequent agent actions, which includes specifications for datasets, task types and evaluation metrics. After all agents completed their tasks, this agent evaluates the experimental results, utilizing insights from the experiment knowledge base to determine whether a revision loop is necessary.

Knowledge Agent The Knowledge Agent is tasked with acquiring and integrating specialized knowledge tailored to specific user requirements. This agent mainly manages two types of knowledge bases: the prior knowledge base and the experiment knowledge base. The prior knowledge base is enriched with task-specific information extracted from sources such as the Open Graph Benchmark (OGB) leaderboards [1], the PyTorch Geometric (PyG) documentation [2], and other online resources, ensuring the agent remains at the cutting edge of technology and methodology. The experiment knowledge base archives detailed experimental outcomes, including models setups and their performance on

[1]https://ogb.stanford.edu/docs/leader_overview/

[2]https://pytorch-geometric.readthedocs.io/en/latest/

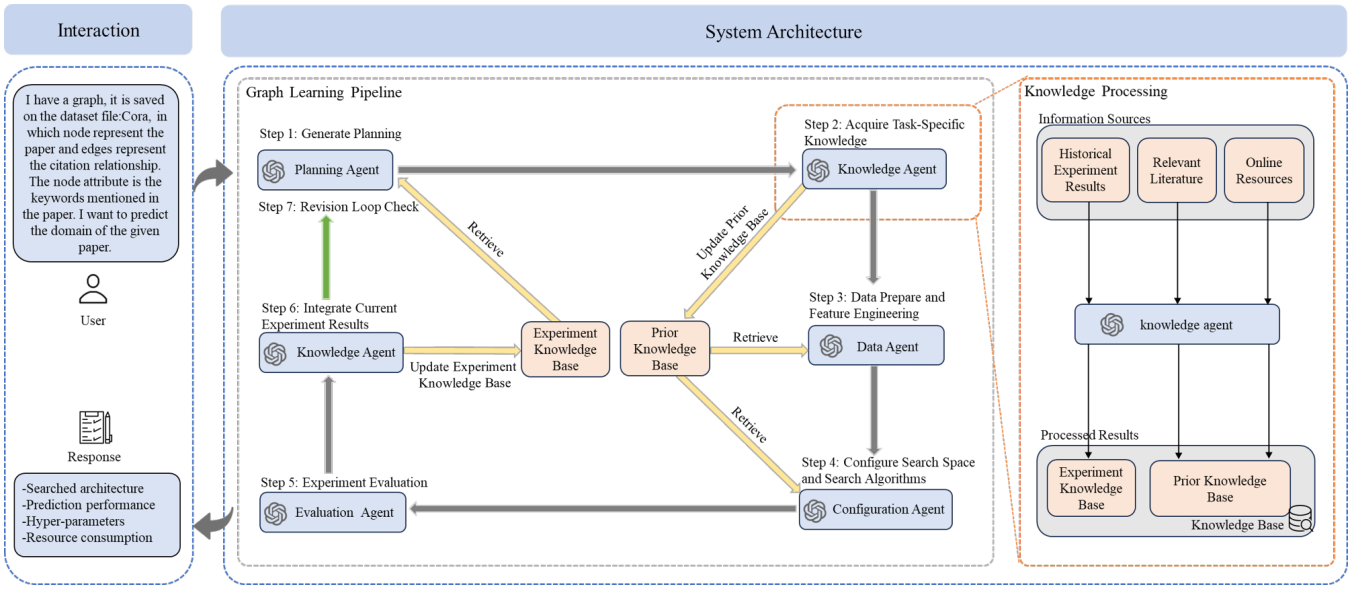

Figure 1: System architecture of LLMGraph. Through a knowledge-driven approach, agents interact with knowledge bases to gain deep insights into tasks and graphs, thereby enhancing the autonomous graph learning capabilities of LLMGraph.

specific datasets, thereby providing insights into their effectiveness and application contexts.

Data Agent The Data Agent utilizes insights from the prior knowledge base to perform feature engineering tailored to specific graphs and tasks, ensuring alignment with expert practices in a knowledge-driven manner.

Configuration Agent The Configuration Agent is responsible for configuring the search space, which includes possible model architecture configurations such as layers and connections, and the search algorithm that explores this space. It interacts with the prior knowledge base to gain insights on model design, enhancing the effectiveness of search space configuration and algorithm selection.

Evaluation Agent The Evaluation Agent is designed to conduct experiments to validate the proposed solutions. After completing the experiments, the Evaluation Agent transmits the results to the Knowledge Agent for integration into the experiment knowledge base.

Demonstration

In this section, we demonstrate the use case of LLMGraph on a real-world problem. For example, users aim to predict the category of articles within a citation network.

As shown in Figure 2, **(a)** illustrates the user's input instructions, **(b)** displays the system's experimental results, achieving an accuracy of 0.8680 on the Cora dataset, surpassing the GNN-based baselines GCN (Kipf and Welling 2016) at 0.8568, ACM-GCN (Luan et al. 2022) at 0.8667, and the AutoML-based baseline SANE (Zhao, Yao, and Tu 2021) at 0.8640. **(c)** displays the task plan generated by the Planning Agent, which interprets the user's intent to predict the category of articles within a citation network as a node classification task. The Planning Agent assigns the appropriate evaluation metrics and identifies the user's preference

Figure 2: This figure demonstrates LLMGraph's ability to handle real-world problem, showcasing its feasibility in task understanding, experimental evaluation, and knowledge-driven design of solutions.

for the neural network model structure. **(d)** shows the Data Agent retrieve relevant information from the prior knowledge base. Specifically, the Data Agent extracts methods related to node classification tasks from the prior knowledge base. This includes specific knowledge on feature engineering from the PyG documentation and practical experiences in data handling archived in related literature. Utilizing this knowledge base, the agent assesses and integrates various feature engineering techniques that are suited for the specific characteristics of the graph data at hand. This knowledge-driven process improves the effectiveness of LLMGraph in handling graph learning problems.

This demonstration showcases the feasibility and effectiveness of LLMGraph in customizing graph learning solutions for real-world problem. For more details, please refer to the appendix of extended version.

References

Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901.

Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a graph: Encoding graphs for large language models. *arXiv preprint arXiv:2310.04560*.

Gilmer, J.; Schoenholz, S. S.; Riley, P. F.; Vinyals, O.; and Dahl, G. E. 2017. Neural Message Passing for Quantum Chemistry. In *ICML*, 1263–1272.

Kipf, T. N.; and Welling, M. 2016. Semi-supervised classification with graph convolutional networks. *ICLR*.

LeCun, Y. 2022. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1): 1–62.

Li, Y.; Li, Z.; Wang, P.; Li, J.; Sun, X.; Cheng, H.; and Yu, J. X. 2023. A survey of graph meets large language model: Progress and future directions. *arXiv preprint arXiv:2311.12399*.

Liu, Z.; Yu, X.; Fang, Y.; and Zhang, X. 2023. Graphprompt: Unifying pre-training and downstream tasks for graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 417–428.

Lü, L.; Medo, M.; Yeung, C. H.; Zhang, Y.-C.; Zhang, Z.-K.; and Zhou, T. 2012. Recommender systems. *Physics reports*, 519(1): 1–49.

Luan, S.; Hua, C.; Lu, Q.; Zhu, J.; Zhao, M.; Zhang, S.; Chang, X.-W.; and Precup, D. 2022. Revisiting Heterophily For Graph Neural Networks. In *NeurIPS*.

Sun, M.; Zhou, K.; He, X.; Wang, Y.; and Wang, X. 2022. Gppt: Graph pre-training and prompt tuning to generalize graph neural networks. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 1717–1727.

Wang, H.; Feng, S.; He, T.; Tan, Z.; Han, X.; and Tsvetkov, Y. 2024. Can language models solve graph problems in natural language? *Advances in Neural Information Processing Systems*, 36.

Zhang, X.; Wang, L.; Helwig, J.; Luo, Y.; Fu, C.; Xie, Y.; Liu, M.; Lin, Y.; Xu, Z.; Yan, K.; et al. 2023. Artificial Intelligence for Science in Quantum, Atomistic, and Continuum Systems. *arXiv preprint arXiv:2307.08423*.

Zhao, H.; Yao, Q.; and Tu, W. 2021. Search to aggregate neighborhood for graph neural network. In *ICDE*.